

Unit – 3 PL/SQL Exceptions [5 Hrs]

Exceptions Handling in PL/SQL

An error occurring during runtime affecting normal flow of execution of a program is called **exception**. Process of handling such errors/exception during execution of program is called **exception handling**.

PL/SQL facilitates programmers to catch such conditions using exception block in the program and an appropriate action is taken against the error condition.

General syntax for exception handling:

```
DECLARE
    declarations section;

BEGIN
    executable command(s);

EXCEPTION
    WHEN exception1 THEN
        statement1;
    WHEN exception2 THEN
        statement2;
    [WHEN others THEN]
        /* default exception handling code */

END;
```

PL/SQL Pre-defined Exceptions

There are many pre-defined exception in PL/SQL which are executed when any database rule is violated by the programs.

Exception	Description
ACCESS_INTO_NULL	It is raised when a NULL object is automatically assigned a value.
CASE_NOT_FOUND	It is raised when none of the choices in the "WHEN" clauses of a CASE statement is selected, and there is no else clause.
COLLECTION_IS_NULL	It is raised when a program attempts to apply collection methods other than exists to an uninitialized nested table or varray, or the program attempts to assign values to the elements of an uninitialized nested table or varray.
DUP_VAL_ON_INDEX	It is raised when duplicate values are attempted to be stored in a column with unique index.

INVALID_CURSOR	It is raised when attempts are made to make a cursor operation that is not allowed, such as closing an unopened cursor.
INVALID_NUMBER	It is raised when the conversion of a character string into a number fails because the string does not represent a valid number.
LOGIN_DENIED	It is raised when a program attempts to log on to the database with an invalid username or password.
NO_DATA_FOUND	It is raised when a select into statement returns no rows.
NOT_LOGGED_ON	It is raised when a database call is issued without being connected to the database.
PROGRAM_ERROR	It is raised when PL/SQL has an internal problem.
ROWTYPE_MISMATCH	It is raised when a cursor fetches value in a variable having incompatible data type.
SELF_IS_NULL	It is raised when a member method is invoked, but the instance of the object type was not initialized.
STORAGE_ERROR	It is raised when PL/SQL ran out of memory or memory was corrupted.
TOO_MANY_ROWS	It is raised when a SELECT INTO statement returns more than one row.
VALUE_ERROR	It is raised when an arithmetic, conversion, truncation, or size-constraint error occurs.
ZERO_DIVIDE	It is raised when an attempt is made to divide a number by zero.

Example of Zero Divide Exception

```

DECLARE
    a INT:=10;
    b INT:=0;
    res INT;
BEGIN
    res:=a/b;
    DBMS_OUTPUT.PUT_LINE('Result is: ' || res);
    NULL;
EXCEPTION
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('Divide by zero is not possible');
END;
/

```

Output:

```

Statement processed.
Divide by zero is not possible

```

NO DATA FOUND:

It is raised WHEN a SELECT INTO statement returns *no* rows. For eg:
Consider the following student table:

SID	NAME
101	Ram
102	Shyam

```
DECLARE
    sname student.name%TYPE;
BEGIN
    SELECT name
    INTO sname
    FROM student WHERE sid=103;
    DBMS_OUTPUT.PUT_LINE('Name:' || sname);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No data to display');
END;
/
```

Output:

```
Statement processed.
No data to display
```

TOO MANY ROWS:

It is raised WHEN a SELECT INTO statement returns more than one row.

```
DECLARE
    sname student.name%TYPE;
BEGIN
    SELECT name
    INTO sname
    FROM student;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No data to display');
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Too many rows selected.');
```

```
END;
/
```

Output:

```
Statement processed.
Too many rows selected.
```

VALUE_ERROR:

This error is raised WHEN a statement is executed that resulted in an arithmetic, numeric, string, conversion, or constraint error. This error mainly results from programmer error or invalid data input.

Example -1

```
DECLARE
  a NUMBER;  --declare a as number
BEGIN
  a:='Ram';  --putting string in a
EXCEPTION
  WHEN VALUE_ERROR THEN
    DBMS_OUTPUT.PUT_LINE('Invalid value to number type');
END;
/
```

Output:

```
Statement processed.
Invalid value to number type
```

Example – 2

```
DECLARE
  sname NUMBER;  --number?
BEGIN
  SELECT name
  INTO sname
  FROM student WHERE sid=102;
EXCEPTION
  WHEN VALUE_ERROR THEN
    DBMS_OUTPUT.PUT_LINE('Invalid value to store');
END;
/
```

Output:

```
Statement processed.
Invalid value to store
```

User Defined Exceptions

This type of exception is called a user-defined exception because the programmer defines it. As a result, before the exception can be used, it must be declared.

These exceptions can be raised explicitly using RAISED keyword.

Syntax:

```
DECLARE
    exception_name EXCEPTION;

BEGIN ...

    IF CONDITION THEN
        RAISE exception_name;

ELSE ...

    END IF;
EXCEPTION

WHEN exception_name THEN ERROR-PROCESSING STATEMENTS;

END;
```

Example is shown below:

```
DECLARE
    age NUMBER:=&age; --taking value at runtime
    invalid_age EXCEPTION; --declaring exception
BEGIN
    --condition for exception
    IF age<18 THEN
        RAISE invalid_age;
    ELSE
        DBMS_OUTPUT.PUT_LINE('Eligible to vote.');
```

Output:

If age = 20 then,
Statement processed.
Eligible to vote.

If age = 16 then,
Statement processed.
Exception - Not Eligible to vote.

Exception Scope

Scope rules in exception handling:

1. We can't DECLARE an exception twice but we can DECLARE the same exception in **two different blocks**.
2. Exceptions declared inside a block are local to that block and global to all its sub-blocks.

As a block can reference only local or global exceptions, enclosing blocks cannot reference exceptions declared in a sub-block.

If we re-declare a global exception in a sub-block, the local declaration prevails i.e. the scope of local is more.

Example – 1 Declaring the same exception in two different blocks.

```
DECLARE
  num INT;
  invalid_num EXCEPTION;
BEGIN

  DECLARE
    invalid_num EXCEPTION;
  BEGIN
    num:=-1;
    IF num<0 THEN
      RAISE invalid_num;
    END IF;
  EXCEPTION
    WHEN invalid_num THEN
      DBMS_OUTPUT.PUT_LINE('Exception-Invalid Number-Must be
Positive');
  END;

  --outside inner block
  num:=4;
  IF MOD(num,2)=0 THEN
    RAISE invalid_num;
  END IF;

EXCEPTION
  WHEN invalid_num THEN
    DBMS_OUTPUT.PUT_LINE('Exception-Invalid Number-Must be
Odd');
END;
/
```

Output:

```
Statement processed.
Exception-Invalid Number-Must be Positive
Exception-Invalid Number-Must be Odd
```

Example 2 – Making global exception for sub-blocks

```
DECLARE
    num INT;
    invalid_num EXCEPTION;
BEGIN
    BEGIN
        num:=-1;
        IF num<0 THEN
            RAISE invalid_num;
        END IF;
    EXCEPTION
        WHEN invalid_num THEN
            DBMS_OUTPUT.PUT_LINE('Exception-Invalid Number-Must be
Positive');
        END;
    END;
END;
/
```

Output:

Statement processed.
Exception-Invalid Number-Must be Positive

Example – 3 Block can't reference exception of sub-blocks

```
DECLARE
    num INT;
BEGIN
    DECLARE
        invalid_num EXCEPTION;
    BEGIN
        num:=-1;
        IF num<0 THEN
            RAISE invalid_num;
            -- this can't be referenced by outer block
            -- this can't be handled
        END IF;
    END;
EXCEPTION
    WHEN invalid_num THEN
        DBMS_OUTPUT.PUT_LINE('Exception-Invalid Number-Must be Positive');
END;
/
```

Output:

ORA-06550: line 16, column 13:
PLS-00201: identifier 'INVALID_NUM' must be declared

Exception Propagation

When an exception occurs, PL/SQL looks for an exception handler in the current block e.g., anonymous block, procedure, or function of the exception.

If it does not find a match, PL/SQL propagates the exception to the enclosing block of the current block. PL/SQL then attempts to handle the exception by raising it once more in the enclosing block.

This process continues in each successive enclosing block until there is no remaining block in which to raise the exception.

If there is no exception handler in all blocks, PL/SQL returns an unhandled exception to the application environment that executed the outermost PL/SQL block.

Example -1

A runtime error occurs in the declaration section of the block. If there is no outer block, execution of the program halts, and control is passed to the host environment. Consider the following script:

```
DECLARE
  num INT:='Ram'; --exception at declaration block
BEGIN
  DBMS_OUTPUT.PUT_LINE(num);
EXCEPTION
  WHEN VALUE_ERROR THEN
    DBMS_OUTPUT.PUT_LINE('Exception has been occurred !');
END;
/
```

Output:

```
ORA-06502: PL/SQL: numeric or value error: character to number conversion error
ORA-06512: at line 2
```

As you can see, the assignment statement in the declaration section of the block causes an error. Even though there is an exception handler for this error, the block cannot execute successfully. **Based on this example, you may conclude that *when a runtime error occurs in the declaration section of the PL/SQL block, the exception-handling section of this block cannot catch the error.***

Example – 2

Consider next example with nested blocks, having error in declaration section of inner block. In this case exception is propagated to outer block and if outer block has exception handler then exception can be handled.

```
BEGIN  --Outer block

  DECLARE
    num INT:='Ram';
```



```

        --exception at declaration section of inner block
BEGIN
    DBMS_OUTPUT.PUT_LINE(num);
EXCEPTION
    WHEN VALUE_ERROR THEN
        DBMS_OUTPUT.PUT_LINE('Exception has been occurred - Inner!');
END;

EXCEPTION  -- Exception handler of outer block
    WHEN VALUE_ERROR THEN
        DBMS_OUTPUT.PUT_LINE('Exception has been occurred - Outer!');
END;
/

```

Output:

Statement processed.
Exception has been occurred – Outer!

In this example, the PL/SQL block is enclosed in another block, and the program can complete. This is possible because the exception defined in the outer block is raised when the error occurs in the declaration section of the inner block. **Therefore, you can conclude that *when a runtime error occurs in the declaration section of the inner block, the exception immediately propagates to the enclosing (outer) block.***

Example – 3

Consider another example with nested blocks having exception in inner block. **If there is no exception handler in inner block exception is propagated to outer block.**

```

BEGIN
    DECLARE
        num INT;
    BEGIN
        num:='Ram';  --Exception
        DBMS_OUTPUT.PUT_LINE(num);
    END;

EXCEPTION  -- Exception handler of outer block
    WHEN VALUE_ERROR THEN
        DBMS_OUTPUT.PUT_LINE('Exception has been occurred!');
END;
/

```

Output:

Statement processed.
Exception has been occurred!

Re-Raising an Exception

On some occasions you may want to be able to stop your program if a certain type of error occurs. **In other words, you may want to handle an exception in the inner block and then pass it to the outer block. This process is called reraising an exception.**

```
--Outer Block
DECLARE
    exception1 EXCEPTION;
BEGIN

    --Inner Block
    BEGIN
        RAISE exception1;
    EXCEPTION
        WHEN exception1 THEN
            RAISE exception1;
    END;

EXCEPTION
    WHEN exception1 THEN
        DBMS_OUTPUT.PUT_LINE('Exception has occurred !');
END;
/
```

Output:

Statement processed.
Exception has occurred !

WATCH OUT!

It is important to note that when an exception is reraised in the block that is not enclosed by any other block, the program is unable to complete successfully. Consider the following example:

```
DECLARE
    e_exception EXCEPTION;
BEGIN
    RAISE e_exception;
EXCEPTION
    WHEN e_exception THEN
        RAISE;
END;
```

When run, this example produces the following output:

```
DECLARE
*
ERROR at line 1:
ORA-06510: PL/SQL: unhandled user-defined exception
ORA-06512: at line 7
```

RAISE_APPLICATION_ERROR:

RAISE_APPLICATION_ERROR is a special built-in procedure provided by Oracle. It allows programmers to create meaningful error messages for a specific application. The RAISE_APPLICATION_ERROR procedure works with user-defined exceptions.

Syntax:

```
RAISE_APPLICATION_ERROR(error_number, error_message);
```

*When RAISE_APPLICATION_ERROR executes it returns error message and error code which looks **same as Oracle built-in error**.*

error_number are pre-defined and have negative integer range from -20000 to -20999.

```
DECLARE
    myexp1 EXCEPTION;
    age INT:=17;
BEGIN
    IF age<18 THEN
        RAISE myexp1;
    END IF;
EXCEPTION
    WHEN myexp1 THEN
        RAISE_APPLICATION_ERROR(-20111, 'Exception-Not Eligible !');
END;
/
```

Output:

ORA-20111: Exception-Not Eligible ! ORA-06512: at line 10

Exception_INIT Pragma

The EXCEPTION_INIT pragma allows you to associate an Oracle error number with the name of a user-defined error. After you associate an error name with an Oracle error number, you can reference the error and write a handler for it.

The EXCEPTION_INIT pragma appears in the declaration section of a block as shown:

```
DECLARE
exception_name EXCEPTION;
PRAGMA EXCEPTION_INIT(exception_name, error_code);
```

Notice that the declaration of the user-defined exception appears before the EXCEPTION_INIT pragma where it is used. The EXCEPTION_INIT pragma has two parameters: exception_name and error_code. exception_name is the name of your exception, and error_code is the number of the Oracle error you want to associate with your exception.

error_number are pre-defined and have negative integer range from -20000 to -20999.

```

DECLARE
    myexp1 EXCEPTION;
    PRAGMA EXCEPTION_INIT(myexp1, -20015);
    age INT:=17;
BEGIN
    IF age<18 THEN
        RAISE myexp1;
    END IF;
EXCEPTION
    WHEN myexp1 THEN
        DBMS_OUTPUT.PUT_LINE('Exception - Not Eligible !');
END;
/

```

Output:

Statement processed.
Exception - Not Eligible !

Example - 2

```

DECLARE
    myexp1 EXCEPTION;
    PRAGMA EXCEPTION_INIT(myexp1, -20015);
BEGIN
    FOR i IN 1..5
    LOOP
        DBMS_OUTPUT.PUT_LINE(i);
        IF i=3 THEN
            RAISE myexp1;
        END IF;
    END LOOP;
EXCEPTION
    WHEN myexp1 THEN
        DBMS_OUTPUT.PUT_LINE('Exception has been raised!');
END;
/

```

Output:

Statement processed.
1
2
3
Exception has been raised!